

1.美国航空公司飞机失事事件分析

(1) 导致事故发生的原因是什么？

直接原因计算机导航系统的数据不一致（飞行计算机的数据库与飞行员使用的航空图在编码上存在冲突）导致飞行员输入了错误的单字母计算机指令。他认为自己输入了目的地 Cali 的坐标，但所用的单字母编码在当时的南美洲航空图上与另一个方向相反的城市波哥大的编码相同，导致飞机飞向了错误的方向并撞山。

除此之外还有一些**间接原因**。输入了错误的单字母计算机指令后，计算机错误持续了 66 秒未被检测到或警告，这表明系统缺乏足够的鲁棒性和错误预警机制。另外，飞行员过于信任自动导航驾驶系统，而没有进行双重检查或对比实际航线。机组人员匆忙遵守空中交通管制指令，忽略对错误指令的处理。这些也是导致飞机失事的原因。

(2) 在软件开发过程中应该强调什么事项以便更好地防止类似问题的发生？

1.**强调数据完整性与一致性**。要确保导航数据库中的所有关键数据（如机场编码、坐标）与官方源、航空图以及所有其他相关系统保持严格的一致性。

2.**严格的输入校验**。对于用户的输入，系统应要求用户提供更多信息或进行二次确认，以确保没有歧义或冲突。比如在这个事件中，系统应该要求飞行员在输入坐标后，进一步通过目的地全称、航线和地图确认目的地无误。

3.**错误检测与预警**。系统应具备完善、严格的错误检测和预警机制，在发生故障时能够及时发出不可忽略的警告信息。

4.**严格的系统级测试**。在软件测试阶段，必须模拟各种真实应用场景的边缘情况和压力环境（如数据不一致性等），以验证系统在极端条件下的稳定性。

2.软件修改问题

(1) 为什么已经投入使用的软件会不断被修改？

软件修改是软件生命周期中最长、最昂贵的阶段，其原因主要分为以下四类：

1.**改正性维护（Corrective Maintenance）**：修复在运行中发现的各种错误、缺陷和故障。

2.**适应性维护（Adaptive Maintenance）**：为了使软件能够适应新的环境而进行的修改，例如：操作系统升级、新硬件平台、数据库版本更新、新的外部接口或网络协议等。

3.**完善性维护（Perfective Maintenance）**：根据用户新的需求或反馈，对软件的功能、性能、用户体验和可用性进行改进和增强。

4.**预防性维护（Preventive Maintenance）**：为了提高软件未来的可维护性、可靠性或可读性而进行的修改，如代码重构、优化设计、更新文档等。

(2) 这些修改会带来什么副作用？

1.**可能会引入新的 Bug**。修改的代码可能会破坏原有代码的功能，导致原先正常的功能出错。

2.**增加软件结构复杂度**。不断修改会使软件结构越来越复杂和混乱，更加难以阅读和维护。

3.**影响软件性能**。修改软件可能会因为各种原因导致运行效率降低。

4.**增加时间和经济成本**。软件修改与维护成本难以估量，可能出现预算超支或者无法按时提交的问题。

(3) 有哪些软件工程措施可以防范修改带来的不利影响？

1.**回归测试**：在进行任何修改后，必须运行完整的回归测试，以确保旧功能没有被新代

码破坏。

2.模块化设计：对软件的各部分划分为独立的模块，这样可以对对一个模块的修改对其他模块功能的影响降到最低。

3.版本控制系统：使用 Git 等工具跟踪每一次代码变更，可以方便地审查、合并代码，并在出现问题时快速回滚到之前的稳定版本。

4.代码审查：所有代码变更都必须经过其他开发人员的审查，有助于在代码合并前发现潜在的错误和不合理的设计。

3.软件的技术质量不等于产品质量

一个典型的例子是 2009 年的 Google Wave。Google Wave 在 2009 年的 I/O 大会上首次亮相时，人们发现它是一种与众不同的工具。不仅是因为它是远程办公热潮出现之前的第一个统一办公区和协作平台，而且还在尝试解决如今面临的许多问题。它整合了实时协作编辑、电子邮件、即时通讯等功能，其算法非常先进，保证了多人同时编辑的并发一致性。从技术角度看，Google Wave 具有很高的技术质量。

然而，不到两年的时间，Google Wave 就失败了。尽管技术先进，但普通用户觉得它过于复杂，难以理解。它的核心概念“Wave”对用户来说很陌生，不知道该在什么场景下使用。它试图一次性解决太多问题，反而导致用户体验混乱，学习成本极高。最终，因为它没能清晰地满足用户的某一特定需求，而被市场和用户抛弃。

那么到底什么是高质量的软件产品？我认为，一个高质量的软件产品，首先能够稳定、准确、高效地实现所有用户所需功能，同时具备良好的内部结构、可维护性、可靠性和安全性。一个真正高质量的软件产品是内部质量和外部质量的统一体。

4.分析 JAVA 计算机程序

(1) 正确性 评价：中

程序基本实现了正确的四则运算逻辑，但是还有一些逻辑缺陷。比如对于乘法运算，提示用户输入×，但是使用 `oper.equals("*")`，匹配*符号，如果用户输入×则无法实现功能。这是一个逻辑错误。另外对于除以 0 的情况也没有进行特殊处理。

(2) 可靠性 评价：低

程序通过 `Double.parseDouble` 对字符串转化为浮点数。如果用户在 A 和 B 输入非数字，程序会抛出异常并崩溃。另外如果输入的运算符不是“+、-、*、/”中的任何一个，程序不会进行计算，但也不会有错误提示，缺乏异常处理机制。这些问题说明程序鲁棒性极差。

(3) 可维护性 评价：低

首先，所有的代码（获取输入、逻辑计算、输出结果）都集中在 `main` 方法中，使得代码耦合度高，难以阅读和修改。如果未来需要修改输入的来源（例如从文件读取）或输出的格式，都必须改动整个 `main` 方法。

其次，运算逻辑使用了四次独立的 `if` 语句来判断操作符。虽然功能简单，但运算效率低下，且若增加更多运算符，代码将变得冗长且难以管理。

(4) 可复用性 评价：低

程序的逻辑计算部分与用户交互被紧密耦合在唯一的 `main` 方法中。如果需要在其他应用程序中复用计算功能，必须重写或提取逻辑。应将计算的核心逻辑封装到一个独立的类方法中，使其与 I/O 操作分离。

(5) 可扩展性 评价：低

程序的结构是僵化的。如果想增加一个新的运算功能，比如幂运算（^）或模（%），就必须添加一个新的 `if` 条件分支，并修改提示信息。这违反了软件设计中的对扩展开放，

对修改关闭的原则。在优良的软件设计中，添加新功能应该只需要添加新的模块（例如一个新的运算类），而不需要改动已有的核心逻辑代码。